

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Distance-based Learning for Mixed Data: The manydist Package

Alfonso Iodice D'Enza
University of Naples Federico II

Michel van de Velden
Erasmus University Rotterdam

Angelos Markos
Democritus University of Thrace

Carlo Cavicchia
Erasmus University Rotterdam

Abstract

Computing distances between observations described by mixed-type variables – both numerical and categorical – is a fundamental task in clustering, multidimensional scaling, and nearest-neighbour methods. Existing distance measures can suffer from importance biases: the contribution of a variable may depend on its measurement type, distribution, scaling, or number of categories. We present the R package manydist, which constructs mixed-variable distances that account for such biases. We describe distance construction, diagnostics, and benchmarking, showing how leave-one-variable-out contributions and comparisons between candidate distances make the choice of distance an inspectable part of an analysis rather than a fixed preprocessing decision. We then illustrate how to integrate distance construction into both unsupervised and supervised pipelines. In particular, using a real world mixed data set we show the package can be used to implement clustering and classification pipelines that allow for proper out-of-sample evaluations.

Keywords: distance, mixed data, commensurability, R.

1. Introduction

Measuring the distance or dissimilarity between observations is a fundamental step in clustering, classification, multidimensional scaling (Borg and Groenen 2005), and nearest-neighbour methods. For numerical variables, standard metrics such as Euclidean and Manhattan distance are widely used. For categorical variables, simple matching and frequency-based dissimilarities are common. Real data, however, frequently combine numerical and categorical variables, and defining a distance that accounts for their different scales and measurement properties is not

straightforward.

A persistent difficulty is *importance bias*: the overall distance can be unequally affected by variables because of their type, scale, distribution, or number of categories. The mechanism is easiest to see for the simple matching dissimilarity, under which two observations differ on a categorical variable whenever they fall in different categories. As the number of categories grows, a random pair is more likely to differ, so the variable's expected contribution increases with the number of levels – a nominal variable with many categories can dominate one with few, independently of any relationship to the phenomenon of interest. Gower's distance (Gower 1971), the most widely used mixed-variable construction, rescales each numerical variable to the unit interval by its range; because most observations fall well inside the extremes, the resulting mean pairwise distance is typically smaller than one, which is why Gower tends to favor categorical contributions (Hennig and Liao 2013; Foss *et al.* 2016). Skewed distributions for the numerical variables amplify this effect: a long-tailed numerical variable is compressed further by range scaling, shrinking its contribution even more. These are undesirable factors in the calculation of an overall distance measure: van de Velden *et al.* (2026) describe these problems and formulate a general mixed-variable distance that do not suffer from them. The proposed unbiased mixed-variable distances have two essential properties: *multivariate additivity* and *commensurability*. Multivariate additivity requires the overall distance to be the sum of variable-specific contributions, so that each variable enters the total additively. Additivity alone does not make the contributions comparable, since a variable measured on a wider scale or split into more categories can still dominate the sum. Commensurability adds this missing calibration: the variable-specific distances are defined to be commensurable when the expected contribution of every variable is the same constant, so that no variable is favoured merely by its type, scale, or number of categories. In the general additive formulation, this is achieved by weighting each variable-specific dissimilarity by the reciprocal of its expected value. Together the two properties yield distances that are *unbiased* in the sense that measurement type and scale do not trivially drive a variable's influence.

Common distances such as Euclidean, Manhattan, Hamming, and Gower are independence-based: their variable-specific contributions are constructed without reference to associations among variables. When variables share information, these distances may count the same underlying contrast more than once, introducing redundancy into the distance calculation. In what follows, we use the term association-aware distances for distance measures that explicitly incorporate associations among variables in their construction. For example, Mahalanobis distance accounts for covariance among numerical variables, whereas categorical constructions can define category dissimilarities through total variation between conditional distributions (Ahmad and Dey 2007; van de Velden *et al.* 2024).

The R package **manydist** (Iodice D'Enza *et al.* 2026) implements unbiased mixed-variable distances, independence-based and association-aware, as well as other mixed-variable distances that do not necessarily fall within the framework established in van de Velden *et al.* (2026). This article focuses on the package implementation, variable-level diagnostics and comparison tools, and the seamless integration of distance computations into learning pipelines. The package's main function, `mdist()`, provides a unified entry point that:

- computes variable-wise dissimilarities using a selected numerical preprocessing and categorical dissimilarity measure;
- estimates variable-wise weights that enforce commensurability;

- aggregates the contributions into an additive distance matrix;
- retains the fitted preprocessing steps so that the same distance can be reapplied to new observations without having to re-estimate them.

Beyond distance construction, **manydist** provides leave-one-variable-out (LOVO) diagnostics and pairwise comparisons of candidate distances, integration with **tidymodels** (Kuhn and Wickham 2020) for predictive workflows, and distance-based interfaces to partitioning around medoids (PAM) and spectral clustering.

The remainder of the article begins with a brief overview of related R packages (Section 2). In Section 3 we briefly describe the methodological framework, stating multivariate additivity and commensurability and deriving the weights that equalize expected variable contributions in the numerical and categorical blocks. Section 4 introduces the package interface, centred on `mdist()` and the `MDist` object that stores the fitted preprocessing. Section 5 covers leave-one-variable-out diagnostics and appraisal of candidate distances. Section 6 shows how distances can be embedded in **tidymodels** workflows. Section 7 concludes the paper.

2. Related software

Several R packages provide distances for homogeneous numerical, categorical, or binary data. Table 1 summarizes commonly used tools that serve as building blocks for clustering, multidimensional scaling, and nearest-neighbour methods.

Software	Description
stats::dist() (R Core Team 2024)	Standard distances for numerical matrices including Euclidean, Manhattan and Minkowski.
proxy::dist() (Meyer and Buchta 2023)	Extensible proximity framework with built-in and registered measures.
parallelDist::parDist() (Eckert 2025)	Parallel distance computation for large numerical data sets.
Rfast::Dist() (Papadakis 2025)	High-performance numerical distance computation using compiled routines.
klaR::kmodes() (Weihs *et al.* 2005)	K-modes clustering using matching dissimilarities for categorical data.
arules::dissimilarity() (Hahsler *et al.* 2023)	Dissimilarities for binary incidence and transaction data.
nomclust::nomclust() (Sulc *et al.* 2025)	Frequency-weighted similarities and clustering for nominal data.

Table 1: R packages providing distances or dissimilarities for homogeneous numerical, categorical, or binary data.

For numerical data, **stats::dist()** supplies the standard metrics, while **proxy::dist()** offers an extensible catalogue and user-defined measures. **parallelDist::parDist()** and **Rfast::Dist()** target larger numerical problems with parallel or compiled implementations. For categorical

and binary data, **klaR::kmodes()** uses matching dissimilarities, **arules::dissimilarity()** supports sparse transaction data, and **nomclust::nomclust()** provides frequency-weighted similarities.

Packages for mixed data are summarized in Table 2 and fall into three broad families. The first comprises implementations of Gower's coefficient: **cluster::daisy()** (`gower` is one of the implemented metrics), **FD::gowdis()**, **StatMatch::gower.dist()**, and **gower::gower_dist()**. These are variants of the same underlying construction rather than independent distance measures, sharing its range-standardization scheme and its tendency to give relatively greater influence to categorical variables when many categorical attributes are present or when standardized numerical differences are comparatively small. The second family controls the relative influence of numerical and categorical information through explicit weighting. **clustMixType::kproto()** combines numerical and categorical dissimilarities within the k-prototypes algorithm (Huang 1998), using a weighting parameter λ that may be specified or estimated automatically, while **kmed::distmix()** implements several established mixed-data distances, including the Gower, Podani, Wishart, Huang, Harikumar, and Ahmad formulations. The third family learns an adaptive mixed-data distance from the data. Related to this family, **kamila::kamila()** balances the numerical and categorical contributions through a weighting estimated from the data rather than specified in advance, and **clustrd::cluspcamix()** couples dimension reduction to the clustering objective so that the representation and the partition are chosen together. Both address the balance between variable types as part of a clustering procedure, whereas the constructions developed here fix it in the distance itself, before any algorithm is applied. **kdml::dkss()** constructs a kernel-based distance in which variable-specific bandwidths are estimated from the data and determine each variable's contribution to the resulting metric.

Many existing interfaces for data-adaptive distances compute the distance directly from all observations supplied to them and do not expose separate fitting and application operations. By contrast, for supported distance specifications, **manydist** explicitly separates these operations. Within each resample, the numerical preprocessing, category dissimilarities, and commensurability weights are estimated using only the observations assigned to model fitting and then held fixed when distances are computed for the observations assigned to evaluation. Estimating these quantities once on the complete data set before resampling would allow the evaluation observations to influence the learned distance and would therefore introduce data leakage. Section 6 illustrates this fit-and-apply procedure.

Software	Description
cluster::daisy() (Maechler *et al.* 2025)	Gower dissimilarity for numerical, nominal, ordinal and binary variables.
FD::gowdis() (Laliberté *et al.* 2014)	Gower variants used in ecology and functional diversity analysis.
StatMatch::gower.dist() (D'Orazio *et al.* 2023)	Gower distance for statistical matching and data fusion.
gower::gower_dist() (van der Loo 2025)	High-performance Gower computation and nearest matching.
clustMixType::kproto() (Szepannek 2018)	K-prototypes clustering with numerical and categorical components.
kmed::distmix() (Budiaji 2022)	Multiple mixed-data distances including Wishart, Podani, Huang and Ahmad-Dey.

Software	Description
kdml::dkss() (Thompson and Ghashti 2025)	Kernel-based mixed-data distance using distributional comparisons.
kamila::kamila() (Foss and Markatou 2018)	Mixed-data clustering balancing numerical and categorical contributions by kernel density weighting.
clustrd::cluspca() (Markos *et al.* 2019)	Joint dimension reduction and clustering for mixed-type data.

Table 2: R packages implementing distances or dissimilarities for mixed numerical and categorical data.

The packages reviewed above provide a broad collection of mixed-data distances and specialized distance-based methods: **manydist** complements this ecosystem by treating distance construction as a fitted and inspectable component of an analysis. It combines additive and commensurable distances with association-aware constructions, diagnostics for examining variable contributions and comparing candidate distances, and interfaces for clustering and predictive modeling. For data-adaptive specifications used within resampling, the numerical preprocessing, category dissimilarities, and commensurability weights are estimated from the training data and then held fixed when computing test-to-training distances.

The **manydist** package is available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/package=manydist. The package website, https://alfonsoiodicede.github.io/manydist_package/, provides reference documentation, supporting articles, and reproducible versions of the examples presented below.

3. A unified mixed-variable distance framework

We follow the notation of van de Velden *et al.* (2026). Let I be the number of observations and let $Q = Q_n + Q_c$ be the number of variables, partitioned into Q_n numerical and Q_c categorical variables. Observed mixed, numerical, and categorical vectors are denoted by $\mathbf{x}$, $\mathbf{x}^n$, and $\mathbf{x}^c$; their random counterparts use $\boldsymbol{X}$. The observed $I \times Q$ data matrix is denoted by $\mathbf{X}_{\text{obs}}$.

For observed vectors $\mathbf{x}_i$ and $\mathbf{x}_l$, manydist constructs an overall distance by combining contributions from the numerical and categorical blocks. Using variable-wise dissimilarities δ_j and weights w_j, the general setup is

$$\begin{aligned} d(\mathbf{x}_i, \mathbf{x}_l) &= \sum_{j=1}^{Q} d_j(\mathbf{x}_i, \mathbf{x}_l) \\ &= \sum_{h=1}^{Q_n} w_h \, \delta_h^n(\mathbf{x}_i^n, \mathbf{x}_l^n) + \sum_{k=1}^{Q_c} w_k \, \delta_k^c(\mathbf{x}_i^c, \mathbf{x}_l^c). \end{aligned} \tag{1}$$

The numerical contribution indexed by h can depend on the complete numerical block, and the categorical contribution indexed by k can depend on the complete categorical block. The

present formulation does not use cross-type associations inside these component dissimilarities.

For Q-dimensional observations $\mathbf{x}_i = (x_{i1}, \ldots, x_{iQ})$ and $\mathbf{x}_l = (x_{l1}, \ldots, x_{lQ})$, a function $d : \mathcal{X}^Q \times \mathcal{X}^Q \to [0, \infty)$ is *multivariate additive* if

$$d(\mathbf{x}_i, \mathbf{x}_l) = \sum_{j=1}^{Q} d_j(\mathbf{x}_i, \mathbf{x}_l), \tag{2}$$

where $d_j(\mathbf{x}_i, \mathbf{x}_l)$ is the contribution associated with variable j. If every d_j is a metric on $\mathcal{X}^Q$, their sum is also a metric (see van de Velden *et al.* 2026, for the proof).

For the numerical block, the additive construction uses weighted Manhattan contributions on possibly transformed coordinates:

$$d_{\text{num}}(\mathbf{x}_i, \mathbf{x}_l) = \sum_{h=1}^{Q_n} w_h \left| f_h(\mathbf{x}_i) - f_h(\mathbf{x}_l) \right|, \tag{3}$$

where f_h may be a univariate or multivariate transformation. Common transformations of the starting values may be required to deal with different measurement scales:

- **standard-deviation scaling:** $(x - \bar{x})/s_x$, which uses all observations rather than only the extremes;
- **range scaling:** $(x - x_{\min})/(x_{\max} - x_{\min})$, as in Gower's distance, which preserves the distributional shape but, for the non-commensurable presets, is sensitive to outliers through the extremes;
- **robust scaling:** $(x - x_{\text{Med}})/(x_{Q_3} - x_{Q_1})$, which replaces the mean and standard deviation with the median and interquartile range and so resists outliers in those same presets.

For categorical variable k with q_k categories, let $\mathbf{z}_{ik}$ denote the q_k-dimensional indicator vector identifying the category observed for individual i. This vector contains one entry equal to one, corresponding to the observed category, and all remaining entries equal to zero. Stacking these vectors gives the $I \times q_k$ indicator matrix $\mathbf{Z}_k$.

Let $\mathbf{\Delta}_k$ be a $q_k \times q_k$ matrix whose entry $\delta_{k,ab}$ specifies the dissimilarity between categories a and b. Typically, $\delta_{k,aa} = 0$ and $\delta_{k,ab} = \delta_{k,ba}$. The observation-level dissimilarity matrix for variable k is then

$$\mathbf{D}_k = \mathbf{Z}_k \mathbf{\Delta}_k \mathbf{Z}_k^\top. \tag{4}$$

Its (i, l)th entry is

$$[\mathbf{D}_k]_{il} = \mathbf{z}_{ik}^\top \mathbf{\Delta}_k \mathbf{z}_{lk} = \delta_{k,ab}, \qquad \text{when } x_{ik} = a \text{ and } x_{lk} = b. \tag{5}$$

Because each indicator vector selects exactly one row and one column of $\mathbf{\Delta}_k$, the (i, l)th entry of $\mathbf{D}_k$ is simply the dissimilarity assigned to the categories held by observations i and l. This representation separates category membership, encoded by $\mathbf{Z}_k$, from the rule used to compare categories, encoded by $\mathbf{\Delta}_k$. For simple matching, all distinct categories have dissimilarity one;

in association-aware constructions, the entries of $\mathbf{\Delta}_k$ are instead estimated from relationships with other variables or with a response.

Consequently, the additive contribution of the complete categorical block is

$$\begin{aligned} d_c(\mathbf{x}_i, \mathbf{x}_l) &= \sum_{k=1}^{Q_c} w_k \, \delta_k^c(\mathbf{x}_i^c, \mathbf{x}_l^c) \\ &= \sum_{k=1}^{Q_c} w_k \, \mathbf{z}_{ik}^\top \mathbf{\Delta}_k \mathbf{z}_{lk}. \end{aligned} \tag{6}$$

3.1. Commensurability

We call a mixed-variable distance *unbiased* when it is multivariate additive and its variable contributions are *commensurable*, so their expected magnitudes do not depend on measurement type or scale.

Let $\boldsymbol{X}_i = (X_{i1}, \ldots, X_{iQ})$ denote the random vector corresponding to observation i. The variable contributions are *commensurable* when, for every j and $i \neq l$,

$$\mathbb{E}[d_j(\boldsymbol{X}_i, \boldsymbol{X}_l)] = c, \tag{7}$$

where the particular value of the constant c is unimportant. Additivity provides an interpretable decomposition; commensurability places those contributions on equivalent expected scales.

For components with positive expected dissimilarity, commensurability in Equation 1 follows from the population weights

$$w_h = \frac{1}{\mathbb{E}[\delta_h^n(\boldsymbol{X}_i^n, \boldsymbol{X}_l^n)]}, \qquad w_k = \frac{1}{\mathbb{E}[\delta_k^c(\boldsymbol{X}_i^c, \boldsymbol{X}_l^c)]}. \tag{8}$$

In practice, manydist estimates these expectations from the observed pairs, provided the resulting component means are positive:

$$\begin{aligned} m_h &= \frac{1}{I(I-1)} \sum_{i \neq l} \delta_h^n(\mathbf{x}_i^n, \mathbf{x}_l^n), \qquad w_h = \frac{1}{m_h}, \\ m_k &= \frac{1}{I(I-1)} \sum_{i \neq l} \delta_k^c(\mathbf{x}_i^c, \mathbf{x}_l^c), \qquad w_k = \frac{1}{m_k}. \end{aligned} \tag{9}$$

Thus, each variable-wise distance is divided by its empirical mean before the contributions are combined.

For a numerical variable h, the location-scale transformations introduced above are vacuous once variable-wise commensurability is applied. Each has the form $f_h(x) = (x - a_h)/b_h$, so the offset cancels in the difference and the scale factor cancels between $|f_h(\mathbf{x}_i) - f_h(\mathbf{x}_l)|$ and m_h;

the three yield identical distances, and identical to the untransformed variables. The scaling choice is therefore consequential only for the non-commensurable settings.

For categorical variable k, let $\boldsymbol{p}_k$ denote the vector of category probabilities. Its expected unweighted contribution is

$$\mathbb{E}[\delta_k^c(\boldsymbol{X}_i^c, \boldsymbol{X}_l^c)] = \boldsymbol{p}_k^\top \boldsymbol{\Delta}_k \boldsymbol{p}_k, \tag{10}$$

which depends on the number and prevalence of categories and on $\boldsymbol{\Delta}_k$. For simple matching, for example, $\boldsymbol{\Delta}_k = \mathbf{1}\mathbf{1}^\top - \mathbf{I}$ and the expected contribution reduces to $1 - \boldsymbol{p}_k^\top \boldsymbol{p}_k$, which for equally likely categories equals $(q_k - 1)/q_k$. More generally, categorical variables need not have the same expected contribution even when they are measured on the same nominal scale. Differences in their category distributions and in the dissimilarity construction itself can induce unequal contributions; see van de Velden *et al.* (2026) for a more detailed treatment. Simple matching provides a particularly transparent example: its expected contribution increases toward one as the number of categories grows, so a matching variable with many levels contributes more than one with few. The weights in Equation 9 correct these differences in expected contribution and thereby restore commensurability.

3.2. Association-aware distances

Commensurability corrects differences in the magnitude of the variable contributions, but it does not account for information shared by several variables. The general formulation in Equation 1 does permit incorporation of such associations as a contribution indexed by variable j may depend on the complete same-type vectors $\mathbf{x}_i^n, \mathbf{x}_l^n$ or $\mathbf{x}_i^c, \mathbf{x}_l^c$ rather than only on the values for variable j. We refer to dissimilarities that incorporate this joint multivariate structure as association-aware distances.

For numerical variables, association awareness can be introduced through principal-component scaling, as described in van de Velden *et al.* (2026). The numerical variables are first transformed to principal-component coordinates, so that the resulting contributions reflect the dependence structure among the original variables. Applying commensurability separately to the transformed coordinates gives each retained component the same expected contribution to the overall distance.

This component-wise scaling may, however, give relatively large influence to principal components that explain little variation in the original data. To accommodate this case, **manydist** also provides a groupwise commensurability option, in which the numerical block is scaled as a whole rather than equalizing the contribution of each transformed coordinate separately. This preserves the relative importance of the principal components while placing the overall numerical block on a scale comparable with the categorical contribution.

The category-dissimilarity matrix provides a common representation for independence- and association-based constructions, following van de Velden *et al.* (2024). For categorical variables, association enters through conditional distributions rather than a covariance matrix. For an association-based construction, assuming every retained category is observed, let $\mathbf{R}^{k'|k} = (\mathbf{Z}_k^\top \mathbf{Z}_k)^{-1} \mathbf{Z}_k^\top \mathbf{Z}_{k'}$ be the conditional distribution of variable k' given the categories of variable k. Dissimilarities between two categories a and b of variable k compare the corresponding

rows $\mathbf{r}_a$ and $\mathbf{r}_b$, using total variation (Ahmad and Dey 2007) or a symmetric Kullback–Leibler construction (Le and Ho 2005). Under independence, every category of k induces the same conditional distribution of k', so the rows coincide and the association-based category dissimilarities vanish. They increase as the categories become more distinct in what they imply about the other categorical variables. If a variable's dissimilarities vanish identically, its mean in Equation 9 is zero; the reciprocal-mean weight is then undefined, so the implementation must either retain that component as a zero contribution or apply an explicitly stated fallback rule.

An association-based categorical distance whose category dissimilarities are constructed using their associations with a response variable is defined *response-aware*, as proposed by van de Velden *et al.* (2024). In particular, when a response $\mathbf{y}$ is supplied, $\mathbf{R}^{y|k}$ records its distribution within each category of predictor k. The category dissimilarity is then determined based on differences in these distributions across the response-variable classes.

4. Distance construction

The **manydist** package separates the construction of a distance from its subsequent diagnostic and modeling uses. This section introduces distance construction with `mdist()`, including custom specifications, named presets, response-aware distances, and application to new observations. The following sections build on these fitted distance objects for diagnostics and learning pipelines.

4.1. Data preparation

The examples use the Palmer penguins data from the **palmerpenguins** package. We retain four numerical variables, two categorical variables, and the categorical response `species`, which will later be used as a response variable. Observations with missing values are removed. The categorical variables are stored as factors, as required for their categorical treatment by `mdist()`.

```
R> penguins <- palmerpenguins::penguins |>
+  dplyr::select(-year)|>
+  tidyr::drop_na() |>
+  dplyr::mutate(
+    dplyr::across(c(species, island, sex), as.factor)
+  )
```

To illustrate later how a fitted distance can be applied to new observations, we split the data into training and test sets. The response variable is retained in both data frames: it can be excluded when constructing predictor-only distances or supplied through the `response` argument when a response-aware distance is required.

```
R> set.seed(2026)

R> penguin_split <- rsample::initial_split(penguins, prop = 0.75,
+                                          strata = species)
```

```
R> penguin_train <- rsample::training(penguin_split)
R> penguin_test <- rsample::testing(penguin_split)
```

4.2. Constructing a distance with mdist()

The central function `mdist()` constructs distances for numerical, categorical, or mixed-type data. Its primary input, `x`, is the data set from which the distance is constructed. At its most basic, a distance is specified through `x` and three components:

- `method_num`, which determines the preprocessing of the numerical variables;
- `method_cat`, which determines the dissimilarity used for categorical variables; and
- `commensurable`, which determines whether the component contributions are rescaled to have equal expected contributions before they are combined.

The variables in `x` should have the appropriate R classes before calling `mdist()`. Numerical variables, including integer-valued variables, are treated as numerical, whereas categorical variables should be stored as factors. Character variables should therefore be converted to factors when they represent categorical information. Likewise, binary variables coded numerically as 0/1 are treated as numerical unless converted to factors.

The main arguments are summarized below. The default specification uses standard-deviation scaling for the numerical variables, total-variation dissimilarities for the categorical variables, and commensurability.

Argument	Role	Default	Main options
`x`	Numerical, categorical, or mixed-type data	required	—
`method_num`	Numerical preprocessing	`"std"`	`"none"`, `"std"`, `"range"`, `"robust"`, `"pc_scores"`
`method_cat`	Categorical dissimilarity	`"tvd"`	`"matching"`, `"tvd"`, and further association-aware, independence-based, and indicator-based methods
`commensurable`	Equalize expected component contributions	`TRUE`	`TRUE`, `FALSE`

Table 3: Main arguments of `mdist()`

The complete set of categorical dissimilarity options can be inspected with `dist_methods_tbl_cat()`. Method-specific arguments control additional aspects of the distance construction, such as the number of principal components retained when `method_num = "pc_scores"`; full details are provided in the package reference documentation.

To illustrate how the different components of `mdist()` determine the resulting distance, consider first Gower's distance. For complete data with equal variable weights, Gower's

construction combines range-scaled absolute differences for numerical variables with matching dissimilarities for categorical variables and averages these contributions across variables. The corresponding unnormalized sum can therefore be obtained as

```
R> library("manydist")

R> d_gower_sum <- mdist(
+  penguin_train |> dplyr::select(-species),
+  method_num = "range", method_cat = "matching", commensurable = FALSE
+)
```

This differs from the usual Gower distance only by the normalization by the number of variables. The same distance on the conventional Gower scale is available directly through the `"gower"` preset:

```
R> d_gower <- mdist(penguin_train |> dplyr::select(-species),preset = "gower")

R> p <- penguin_train |> dplyr::select(-species) |> ncol()

R> all.equal(
+  as.matrix(d_gower_sum$distance) / p,
+  as.matrix(d_gower$distance)
+)

[1] TRUE
```

Thus, the preset provides the normalized Gower distance, corresponding to standard implementations such as those in **cluster** and **gower**.

Other distance specifications can be constructed by changing the numerical preprocessing, categorical dissimilarity, and commensurability settings. For example, the default specification of `mdist()` uses standard-deviation scaling for numerical variables, total-variation dissimilarities for categorical variables, and commensurability.

When `commensurable = TRUE`, any variable-wise affine rescaling of the numerical variables cancels when each contribution is divided by its empirical mean distance. Consequently, choices such as `"std"`, `"range"`, and `"robust"` produce the same training distances under commensurability. Range scaling can differ for new observations outside the training range, since these are clipped to the training limits. Multivariate transformations such as `"pc_scores"` are different because they change the coordinate system rather than rescaling each variable separately.

4.3. Distance presets

Presets either specifiy a predefined combination of `method_num`, `method_cat`, and `commensurable`, or invoke a dedicated implementation of an established mixed-data distance measure. The Gower distance introduced above (`preset = "gower"`) is one of the presets available in `mdist()`; Table 4 provides the complete list.

Preset	Numerical preprocessing	Categorical dissimilarity	Additive	Unbiased	Response-aware
`gower`	range scaling to unit interval	simple matching	yes	no	no
`euclidean`	standard-deviation scaling	sd scaling after indicator (one-hot) encoding	no	no	no
`hl`	standard-deviation scaling	Hennig–Liao indicator scaling	no	no	no
`mod_gower`	modified Gower (Liu *et al.* 2024)	–	yes	no	no
`gudmm`	Generalized Unified Distance Metric (Mousavi and Sehhati 2023)	–	no	no	no
`dkss`	kernel product similarity (Ghashti and Thompson 2024)	–	no	no	no
`u_indep`	standard-deviation scaling	simple matching	yes	yes	no
`u_dep`	principal-component scaling	total variation distance	yes	yes	yes
`u_mix`	standard-deviation scaling	total variation distance	yes	yes	yes

Table 4: Distance implemented via presets. Additive distances satisfy multivariate additivity and unbiased distances additionally enforce commensurability. Response-awareness affects the categorical dissimilarity only and applies when a response variable and an association-aware categorical variant are supplied.

The `u_mix` preset, which is the default, combines standard-deviation scaling for the numerical variables with total-variation dissimilarities for the categorical variables and commensurability:

```
R> d_u_mix <- mdist(penguin_train |> dplyr::select(-species),preset = "u_mix")

R> d_u_mix |> print()
MDist object
  preset : u_mix
  number of observations : 249
  number of continuous variables   : 4
  number of categorical variables   : 2
  parameters:
    - categorical method: tvd
    - numerical preprocessing: std
    - commensurability adjustment: TRUE
```

4.4. Supervised distances

In supervised setting, when a categorical response, or label, is available, `mdist()` can compute response-aware distances through the definition of the `response` argument. For example, using `species` as the response in the Palmer penguins data,

```
R> d_response <- mdist(penguin_train,response = species,preset = "u_mix")
```

constructs a distance in which differences between categories of the categorical predictors can depend on their conditional distributions over `species`. Note that the response is not used in the computation for the numerical part of the distance.

The `mdist()` function can also be used to calculate distances of new observations to the training data. This can be useful in distance-based classification methods like kNN or simply for assigning new data according to existing clusters. The `new_data` argument computes ~~rectangular~~ dissimilarities between the new observations and the training observations using the weights and the scaling quantities estimated from the training data.

```
R> d_response_new <- mdist(
+   penguin_train,
+   response = species,
+   new_data = penguin_test |> dplyr::select(-species),
+   preset = "u_dep"
+)
```

Note that `species` is not supplied for `penguin_test`: the response information entering the distance construction is estimated from the training data only. Thus, the category dissimilarities, commensurability weights, and any fitted numerical transformation are estimated from `penguin_train` and subsequently applied unchanged to `penguin_test`. This fit-and-apply distinction is particularly important in predictive settings, where estimating the distance construction using the evaluation observations would introduce data leakage.

5. Distance diagnostics and benchmarking

With many plausible distance specifications available for mixed data, it is important to examine how dissimilarities depend on individual variables and how differences propagate to downstream analyses. The **manydist** package provides diagnostics at two levels. The first considers direct variable specific effects on the dissimilarity matrices whereas the second considers the consequences for the distance-based downstream procedures classical MDS and clustering.

5.1. Leave-one-variable-out diagnostics

Leave-one-variable-out (LOVO) diagnostics for individual distance specifications are available through the `lovo_mdist()` function. This function first computes the distance using all variables and then removes each variable in turn, recomputes the distance, and quantifies the resulting change. The function can be called using the same inputs as `mdist()`.

Diagnostics corresponding to subsequent analysis can be obtained by specifying the corresponding analysis and involved parameters (e.g, dimensionality of the mds solution, the clustering algorithm and the number of cluster).

To illustrate this functionality, consider the penguins data set and the gower distance. Leave-one-variable-out results can be obtained using:

```
R> lovo_gower <- lovo_mdist(
+  penguin_train |> dplyr::select(-species),
+  preset = "gower"
+)
```

Direct distance diagnostics are:

- `mad_importance` is the mean absolute difference between the full dissimilarity matrix and the matrix obtained after leaving out a variable.
- `relative_distance` is the normalized version of `mad_importance`, allowing the relative LOVO contributions of the variables to be compared on a common scale.

$$\text{relative_distance}_j = \frac{\text{mad_importance}_j}{\sum_{r=1}^{Q} \text{mad_importance}_r}.$$

Thus, the values sum to one across variables and can be interpreted as the relative LOVO contribution of each variable to the change in the overall distance.

The downstream analysis-based diagnostics quantify what the change in distance does to a particular subsequent analysis:

- `mds_congruence` (equivalently, `cc_importance`) and `ac_importance` compare classical MDS representations of the full and reduced distances.
- When clustering is requested, `pam_importance`, `hclust_importance`, or `spectral_importance` measure the change in the corresponding clustering partition as one minus the adjusted Rand index (ARI).

The computed diagnostics measures are stored in the `results` tibble returned by `lovo_mdist()`

```
R> lovo_gower$results
```

```
# A tibble: 6 x 5
  variable         variable_type mad_importance mad_normalized relative_distance
  <chr>            <chr>                  <dbl>          <dbl>             <dbl>
1 island           categorical           0.0849          0.289             0.289
2 bill_length_mm   numeric               0.0354          0.121             0.121
3 bill_depth_mm    numeric               0.0317          0.108             0.108
4 flipper_length_~ numeric               0.0300          0.102             0.102
5 body_mass_g      numeric               0.0307          0.105             0.105
6 sex              categorical           0.0808          0.275             0.275
```

A visual summary of the leave-one-variable-out results on the dissimilarities can be obtained using the `autoplot` function from `ggplot2` and the measure (i.e., either "mad_importance", or, the default, "relative_distance"):

```
R> ggplot2::autoplot(
+   lovo_gower,
+   metric = "relative_distance",
+   reorder = TRUE
+)
```

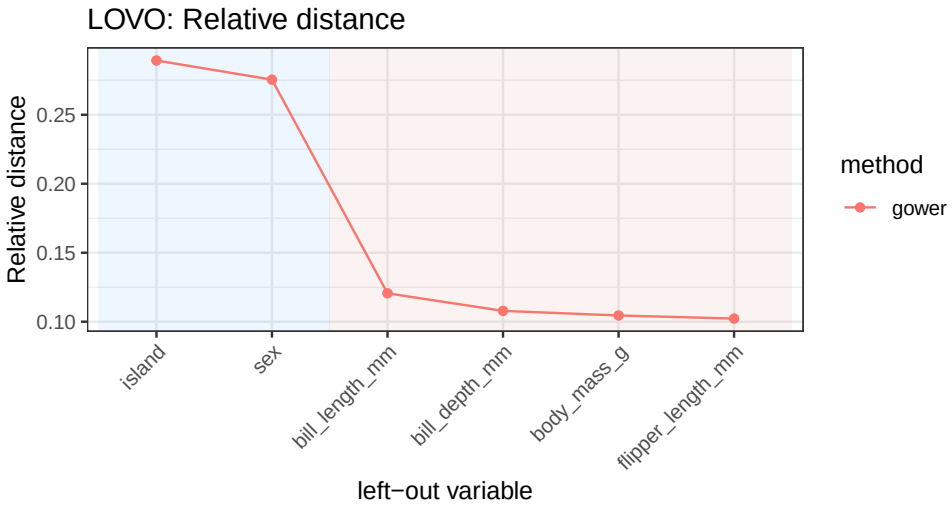


Figure 1: Relative leave-one-variable-out contributions under Gower distance. Larger values indicate a larger relative change in the pairwise dissimilarities when the variable is omitted.

These MDS diagnostics should be distinguished direct distance diagnostics. A variable can change the original dissimilarity matrix substantially without producing a comparable change in a low-dimensional MDS representation, or conversely. The quantities do not provide competing estimates of the same effect: they describe different objects. MDS-based diagnostics are therefore most relevant when such a representation is itself of interest.

The impact of leaving a variable out on cluster analysis can be assessed by specifying the number of clusters and one or more of the following distance based clustering algorithms: hierarchical clustering (`hclust` with complete linkage), partitioning around medoids (`pam`) and spectral clustering (`spectral`, with some defaults?). The effect of leaving a variable out is assessed by calculating the adjusted rand index (ARI; reference) between the clusters obtained by using all variables to the solution obtained by leaving-one-variable out. The clustering importance is defined as 1-ARI so that higher values correspond to more impact on the solution.

For example,

```
R> lovo_gower <- lovo_mdist(
+   penguin_train |> dplyr::select(-species),
+   preset = "gower",
+   mds = TRUE,
+   cluster_methods = c("hclust", "pam", "spectral"),
+   cluster_k = 3
+)
```

The `results` field of `lovo_gower` contains the numerical LOVO diagnostics. Visualizations can be obtained with `autoplot()` by specifying the desired metric. Figure 2 compares the effects on the MDS configuration and the hierarchical clustering partition:

```
R> a <- ggplot2::autoplot(
+   lovo_gower,
+   metric = "ac_importance"
+)

R> b <- ggplot2::autoplot(
+   lovo_gower,
+   metric = "hclust_importance"
+)


R> variable_labels <- c(
+   island = "Island", sex = "Sex",
+   bill_length_mm = "Bill\nlength",
+   bill_depth_mm = "Bill\ndepth",
+   flipper_length_mm = "Flipper\nlength",
+   body_mass_g = "Body\nmass"
+)

R> a <- a + ggplot2::labs(
+   title = "(a) MDS configuration", y = "Alienation coefficient", x = NULL
+)
R> b <- b + ggplot2::labs(
+   title = "(b) Hierarchical clustering", y = "1 - ARI", x = NULL
+)

R> lovo_plot_style <- list(
+   ggplot2::scale_x_discrete(labels = variable_labels),
+   ggplot2::scale_y_continuous(limits = c(0, NA)),
+   ggplot2::theme(
+     legend.position = "none",
+     plot.title = ggplot2::element_text(size = 10, face = "plain"),
+     axis.title = ggplot2::element_text(size = 9),
+     axis.text = ggplot2::element_text(size = 8),
+     axis.text.x = ggplot2::element_text(angle = 0, hjust = 0.5),
+     panel.grid.minor = ggplot2::element_blank()
+   )
+)
R> a <- a + lovo_plot_style
R> b <- b + lovo_plot_style


R> patchwork::wrap_plots(a, b, nrow = 1)
```

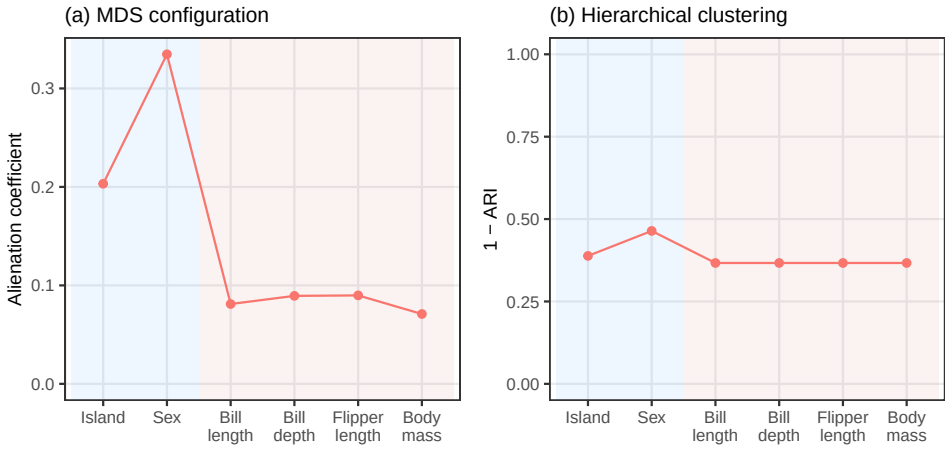


Figure 2: Downstream leave-one-variable-out diagnostics under Gower distance for the penguin training data. (a) Alienation coefficient comparing the two-dimensional classical MDS configurations before and after omitting each variable. (b) One minus the adjusted Rand index (ARI) comparing the full and reduced complete-linkage hierarchical clustering partitions, each with three clusters.

Compare LOVO diagnostics across distance specifications

To compare variable-specific effects across alternative distance specifications, the **manydist** package provides the compare_lovo_mdist() function. The function takes a list of distance specifications and, when relevant, downstream analysis settings. For example,

```
R> variants <- list(
+  Gower = list(preset = "gower"),
+  Euclidean = list(preset = "euclidean"),
+  HL = list(preset = "hl"),
+  `Unbiased Gower` = list(
+    method_num = "range",
+    method_cat = "matching",
+    commensurable = TRUE
+  )
+)

R> lovo_comparison <- compare_lovo_mdist(
+  penguin_train |> dplyr::select(-species),
+  methods = variants,
+  cluster_methods = "pam",
+  cluster_k = 3
+)

R> ggplot2::autoplot(
+  lovo_comparison,
+  metric = "relative_distance",
```

```
+  reorder = TRUE
+)
```

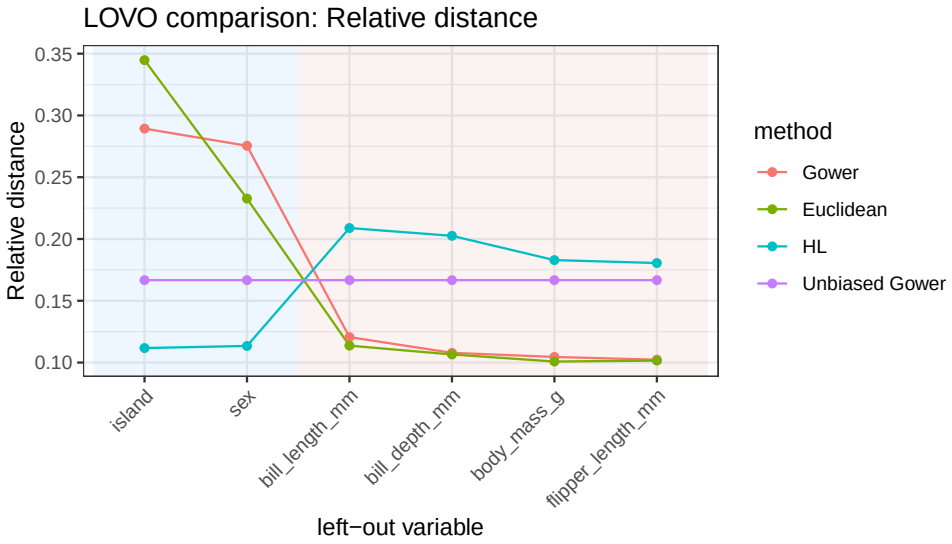


Figure 3: Relative leave-one-variable-out contributions under four distance specifications.

```
R> ggplot2::autoplot(
+  lovo_comparison,
+  metric = "pam_importance",
+  reorder = TRUE
+)
```

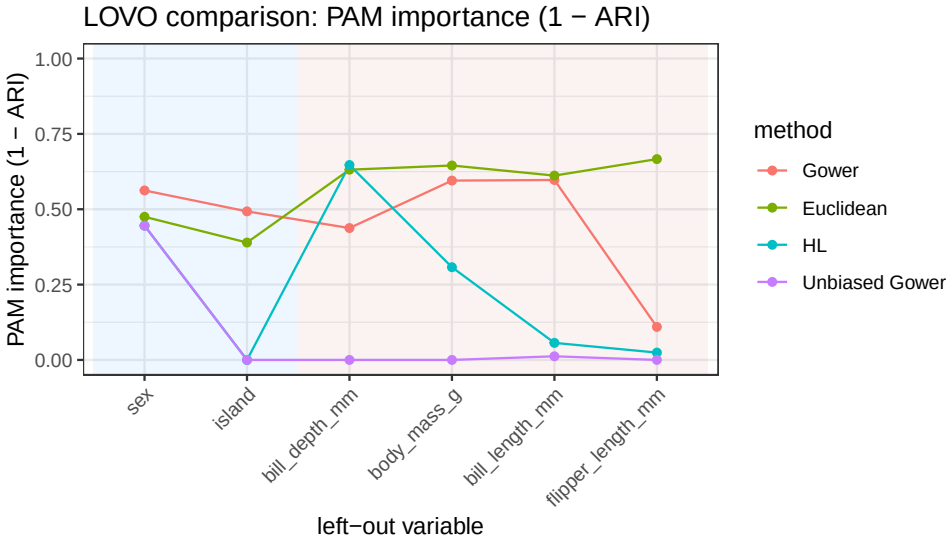


Figure 4: Leave-one-variable-out importance (1 - ARI) for three-cluster PAM solutions under four distance specifications.

Figure 3 compares the relative LOVO contributions across distance specifications, whereas Figure 4 shows the corresponding PAM-based importance. In both cases, each full-versus-

reduced comparison is made within the same distance specification; the species labels are not used. Other diagnostics introduced above can be displayed by changing the `metric` argument.

Note that, for association-based specifications, leaving out a variable can change not only the available information but also the construction of the remaining contributions. Under `u_dep`, for example, removing either categorical variable leaves only one categorical variable. Because the association-based categorical construction requires at least two categorical variables, the reduced specification falls back to matching. LOVO results in such cases should therefore be interpreted with this change in construction in mind.

5.2. Comparing candidate distance specifications

The leave-one-variable-out function can be used to inspect and compare how individual variables impact distances, and possibly subsequent distant-based analyses. One may, however, also be interested in a comparison of the overall distances resulting from different distance specification. Or, how such differences lead to differences in downstream analysis. For this purpose, the function `benchmark_mdist()` can be used.

Candidate distance specifications are described by four fields: `preset`, `method_cat`, `method_num`, and `commensurable`. Named presets determine their settings internally; ~~rows with `preset = "custom"` use the other three fields explicitly~~. By default, `all_dist_method_specs()` returns a table of custom combinations (413 in the current version), filtered through its component arguments. For commensurable specifications, the grid retains only `"std"` and `"pc_scores"` numerical preprocessing to avoid redundant affine scalings. Supplying preset names returns only those presets; `preset = "all"` lists all named presets. Alternatively, the specifications table can be constructed manually. Because `benchmark_mdist()` compares every pair of successful candidates, large candidate sets produce many pairwise comparisons.

If we want to compare the Gower, unbiased, and unbiased preset variants we can do:

```
R> candidate_specs <- all_dist_method_specs(
+  preset = c("gower", "u_indep", "u_dep")
+)

R> set.seed(2026)

R> distance_benchmark <- benchmark_mdist(
+  x = penguin_train |> dplyr::select(-species),
+  specs = candidate_specs,
+  cluster_k = 3,
+  cluster_methods = "pam"
+)
```

The `summary()` method displays one upper-triangular table per benchmark measure, with distance specifications labelling both axes and the diagonal omitted. By default, all available measures are shown; the `metric` argument selects one or more measures (for example, `metric = "relative_distance"` or `metric = "ari_pam"`). The complete tibble, with one row per pair of successful distance specifications, is returned invisibly for further analysis:

```
R> pairwise_results <- summary(distance_benchmark)

Summary of MDistBenchmark
  specifications : 3
  successful     : 3
  failed         : 0
  comparisons    : 3

Pairwise diagnostics (upper triangle; diagonal omitted):

mad:
        gower u_dep u_indep
gower           5.663   5.663
u_dep                   1.440
u_indep

relative_distance:
        gower u_dep u_indep
gower           1.773   1.773
u_dep                   0.239
u_indep

mds_congruence:
        gower u_dep u_indep
gower           0.942   0.992
u_dep                   0.916
u_indep

alienation:
        gower u_dep u_indep
gower           0.336   0.130
u_dep                   0.400
u_indep

ari_pam:
        gower u_dep u_indep
gower           0.481   0.393
u_dep                   0.253
u_indep
```

For two candidate distances d_a and d_b, `mad` is the mean absolute difference between their pairwise dissimilarities. When comparing distance specifications, the overall scale of the distance values can differ considerably, a direct comparison of the absolute difference may therefore not be appropriate. Hence, `relative distance` is also provided, defined as:

$$r(d_a, d_b) = \frac{2\,\mathrm{mean}(|d_a - d_b|)}{\mathrm{mean}(|d_a|) + \mathrm{mean}(|d_b|)}. \tag{11}$$

Thus, `mad` describes the absolute magnitude of the differences between the two dissimilarity matrices, whereas `relative_distance` expresses that difference relative to their overall magnitudes.

Similar to the leave-one-variable-out measures, we assess how different distance specifications lead to different (classical) mds configurations by calculating the ~~the~~ congruence coefficient and its corresponding alienation coefficient.

Clustering comparisons are optional and require specification of the clustering algorithn(s) and the number of clusters. When `cluster_k` is specified, each requested clustering method is applied to every candidate distance, and partitions produced by the same clustering method are compared across pairs of distance specifications using ARI. An ARI of one indicates identical partitions, while lower values indicate increasingly different assignments. These values measure agreement between the resulting partitions; they do not provide any information about the quality of the cluster solutions.

The pairwise results can be extracted with `summary()` as above or displayed using the `autoplot()` method. For example, Figure 5 shows the symmetric relative-distance comparison among the three candidate specifications.

```
R> ggplot2::autoplot(
+   distance_benchmark,
+   metric = "relative_distance"
+)
```

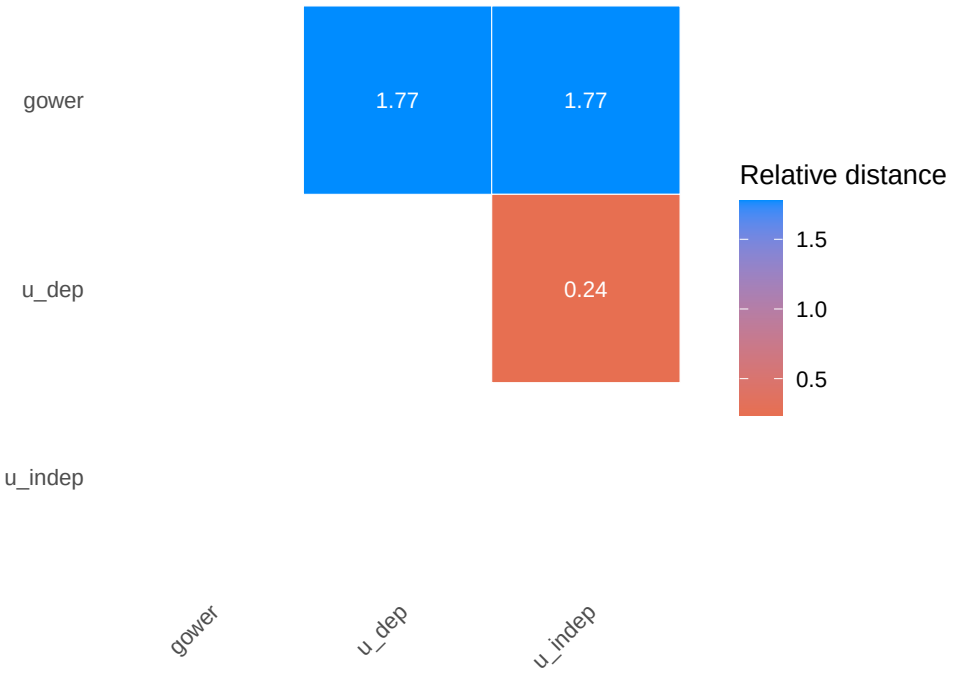


Figure 5: Pairwise symmetric relative distances among three candidate distance specifications. Smaller values indicate more similar dissimilarity matrices after accounting for their overall magnitude.

The same display can be requested for raw MAD, MDS congruence or alienation, and, when clustering has been requested, ARI. Which comparisons are informative therefore depends on how the distance is intended to be used. The diagnostics describe different aspects of candidate distance specifications rather than providing, by themselves, a rule for selecting a preferred distance.

6. Distance-based learning pipelines

The **manydist** package integrates this fit-and-apply principle with **tidymodels**, allowing distance construction to become part of a learning workflow. We begin with a direct nearest-neighbour classifier on tabular data, show the equivalent calculation using precomputed distances, and then express it as a recipe and model workflow. We then illustrate clustering and classification pipelines using the same real-world dataset.

6.1. From tabular data to a distance-based learning workflow

We first predict penguin species using the training and test sets introduced in Section 4. For this direct example, we use `knn_dist()`, with five neighbours and Gower's distance, keeping `species` out of the distance calculation.

The simplest interface supplies the tabular training data in `x` and the test data in `new_data`. The `response` argument selects the training outcome and excludes it from the predictors, including from the test data if present. The function calls `mdist()` internally; `dist_args` supplies its distance settings. Here, `preset = "gower"` uses numerical ranges estimated from the training observations and applies the same preprocessing to the test observations.

```
R> tabular_predictions <- knn_dist(
+  x = penguin_train,
+  response = "species",
+  new_data = penguin_test,
+  k = 5,
+  dist_args = list(preset = "gower"),
+  type = "class"
+)

R> knn_predictions <- tibble::tibble(
+  .pred_class = tabular_predictions
+)

R> dplyr::bind_cols(penguin_test["species"], knn_predictions) |>
+  yardstick::accuracy(truth = species, estimate = .pred_class)

# A tibble: 1 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy multiclass         1
```

Alternatively, the test-to-training distances can be computed externally with `mdist()` and passed directly to `knn_dist()`, together with the training outcomes. No training-to-training matrix needs to be supplied:

```
R> d_knn_test <- mdist(
+  penguin_train |> dplyr::select(-species),
+  new_data = dplyr::select(penguin_test, -species),
+  preset = "gower"
+)
```

The resulting matrix has 84 rows and 249 columns. Its columns correspond to training observations and must be in the same order as the training outcomes. The following call gives the same class predictions as the tabular interface:

```
R> distance_predictions <- knn_dist(
+  x = d_knn_test$distance,
+  y = penguin_train$species,
+  k = 5,
+  type = "class"
+)

R> all.equal(tabular_predictions, distance_predictions)

[1] TRUE
```

For tabular input, this direct interface computes distances on each call rather than returning a fitted model. To combine fitted distance preprocessing and prediction in a reusable pipeline, we use the **parsnip** model specification `nearest_neighbor_dist()` with a **recipes** recipe: an object that specifies preprocessing operations to be estimated from fitting data and applied to new observations. The step `step_mdist()` replaces the original predictors with distances to the training observations. Its `output = "distance_to_training"` setting creates the representation consumed by `nearest_neighbor_dist()`.

```
R> penguin_knn_spec <- nearest_neighbor_dist(
+  mode = "classification",
+  neighbors = 5
+) |>
+  parsnip::set_engine("manydist")

R> penguin_recipe <- recipes::recipe(
+  species ~ .,
+  data = penguin_train
+) |>
+  step_mdist(
+    recipes::all_predictors(),
+    preset = "gower",
+    output = "distance_to_training"
+  )

R> penguin_workflow <- workflows::workflow() |>
+  workflows::add_recipe(penguin_recipe) |>
+  workflows::add_model(penguin_knn_spec)

R> penguin_workflow_fit <- workflows::fit(
+  penguin_workflow,
+  data = penguin_train
+)

R> workflow_predictions <- predict(
+  penguin_workflow_fit,
+  new_data = dplyr::select(penguin_test, -species)
+)

R> all.equal(knn_predictions, workflow_predictions)

[1] TRUE
```

The comparison returns `TRUE`: the workflow gives the same predictions as the direct calculation, while accepting the original predictors as input. During fitting, `step_mdist()` estimates and stores the distance preprocessing and the training predictors. At prediction time, it uses them to construct the required test-to-training distances; test labels are not required.

For supervised resampling, the complete workflow must be fitted separately within each resample. An initial split creates a *training set* and a *test set*, the latter reserved for final

evaluation. Resampling operates within the training set: each resample contains an *analysis set* (the fold-specific training data) and an *assessment set* (the held-out fold). We use training and test for the initial split, and analysis and assessment for the resampling splits. The income-classification application below uses this workflow interface to compare distance specifications and tune the number of neighbours.

6.2. Country clustering

The packaged `wdi_2022` data provide a fixed snapshot of five numerical World Development Indicators and three World Bank classifications for 181 countries (The World Bank 2022). We use these data to compare unsupervised country partitions and then to predict World Bank income group.

```
R> data("wdi_2022", package = "manydist")
```

```
R> wdi_clustering <- wdi_2022 |>
+   dplyr::select(-country)
```

```
R> distance_presets <- c(
+   gower = "gower",
+   euclidean = "euclidean",
+   u_indep = "u_indep",
+   u_dep = "u_dep",
+   u_mix = "u_mix"
+)
```

The clustering workflows contain no response variable, so every candidate distance is constructed from the observed variables alone. To focus on the effects of the distance and clustering method, the number of clusters is fixed at four throughout. This value is illustrative rather than an estimated optimum.

One recipe is defined for each distance specification. Here `output = "pairwise"` instructs `step_mdist()` to provide the square distance matrix required by the clustering method.

```
R> clustering_recipes <- purrr::map(
+   distance_presets,
+   ~ recipes::recipe(~ ., data = wdi_clustering) |>
+     step_mdist(
+       recipes::all_predictors(),
+       preset = .x,
+       output = "pairwise"
+     )
+)
```

Each distance is combined with partitioning around medoids (PAM) and spectral clustering. Both methods use the same pairwise representation produced by the corresponding recipe. For example, the following calls fit both clustering methods using `u_mix`.

```
R> set.seed(2026)
```

```
R> wdi_pam_fit <- generics::fit(
+  pam_dist(num_clusters = 4),
+  recipe = clustering_recipes$u_mix,
+  data = wdi_clustering
+)

R> wdi_spectral_fit <- generics::fit(
+  spectral_dist(num_clusters = 4, nstart = 50),
+  recipe = clustering_recipes$u_mix,
+  data = wdi_clustering
+)
```

Average silhouette width describes separation within each fitted distance–clustering combination. Because each silhouette is calculated using the distance that produced its partition, silhouette values are interpreted within a distance rather than as a criterion for ranking different distance metrics. The ARI in Table 5 instead compares the PAM and spectral assignments under the same distance. Thus, silhouette concerns separation within a fitted solution, whereas ARI concerns agreement between two partitions.

Distance	PAM silhouette	Spectral silhouette	PAM–spectral ARI
Gower	0.416	0.404	0.704
Euclidean	0.271	0.269	0.767
Unbiased independent	0.295	0.324	0.648
Unbiased dependent	0.284	0.275	0.761
Unbiased mixed	0.308	0.341	0.647

Table 5: Four-cluster WDI solutions under five distances and two clustering methods.

The cross-method ARI in Table 5 holds the distance fixed and compares the two clustering methods. The complementary comparison in Figure 6 instead holds the clustering method fixed and compares partitions obtained under different distances. Higher ARI values indicate more similar country assignments.

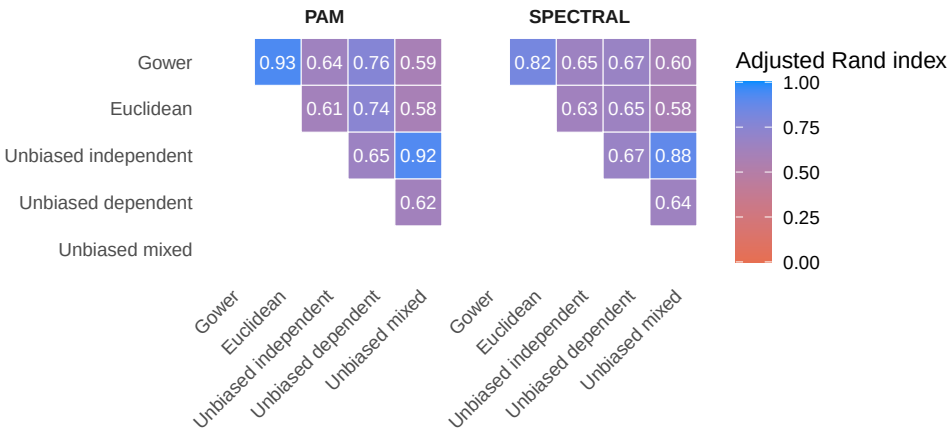


Figure 6: Pairwise adjusted Rand indices for four-cluster WDI partitions. Within each panel, the clustering method is held fixed while the distance varies. Self-comparisons are omitted; colours use a fixed ARI scale from 0 to 1.

Together, these summaries show separately how the clustering assignments vary with the clustering method and with the distance specification. They are descriptive comparisons rather than a rule for choosing a universally preferred clustering solution.

6.3. Supervised income classification

We now predict income group, comparing the five distance specifications under a common resampling design. The recipes use `output = "distance_to_training"` to provide the representation required by `nearest_neighbor_dist()`. Countries labelled `"Not classified"` are removed. Country is an identifier, and lending category is excluded because it is administratively related to income group. The two-country North America level is pooled with Latin America and the Caribbean to reduce the risk that a categorical level is absent from an analysis set.

```
R> wdi_classification <- wdi_2022 |>
+  dplyr::filter(income != "Not classified") |>
+  dplyr::mutate(
+    income = forcats::fct_drop(income),
+    region = forcats::fct_collapse(
+      region,
+      Americas = c(
+        "North America",
+        "Latin America & Caribbean"
+      )
+    )
+  ) |>
+  dplyr::select(-country, -lending)

R> wdi_classification |>
+  dplyr::count(income)
```

```
                income  n
1          High income 63
2           Low income 21
3 Lower middle income 48
4 Upper middle income 47
```

A stratified initial split reserves 25% of the countries as the test set for final evaluation. The remaining training set supplies a common set of five stratified resamples for evaluating every distance and neighbourhood size.

```
R> set.seed(2026)

R> wdi_split <- rsample::initial_split(
+  wdi_classification,
+  prop = 0.75,
+  strata = income
+)

R> wdi_train <- rsample::training(wdi_split)

R> set.seed(2026)

R> wdi_folds <- rsample::vfold_cv(
+  wdi_train,
+  v = 5,
+  strata = income
+)

R> neighbor_grid <- tibble::tibble(
+  neighbors = c(1L, 3L, 5L, 7L, 9L, 15L)
+)
```

Because these recipes include `income` as a response variable, response-aware specifications (`u_dep` and `u_mix`) use the income labels from the current analysis set when their categorical dissimilarities are fitted. Labels from the assessment sets and the final test set are not used in this preprocessing. The other three specifications remain response-independent.

One recipe is defined for each distance and paired with the same nearest-neighbour model. The number of neighbours is tuned over the grid defined above.

The workflow set evaluates the distance specifications under the same resampling design, tuning the number of neighbours within each workflow. Balanced accuracy is the primary metric because the four income groups have different sizes. The box separates preprocessing, model specification, workflow assembly, and tuning.

Income classification: cross-validation

Preprocess

```
R> classification_recipes <- purrr::map(
+  distance_presets,
+  ~ recipes::recipe(income ~ ., data = wdi_train) |>
+    step_mdist(
+      recipes::all_predictors(),
+      preset = .x,
+      output = "distance_to_training"
+    )
+)
```

Specify

```
R> knn_spec <- nearest_neighbor_dist(
+  mode = "classification",
+  neighbors = tune::tune()
+) |>
+  parsnip::set_engine("manydist")
```

Assemble

```
R> classification_workflows <- workflowsets::workflow_set(
+  preproc = classification_recipes,
+  models = list(knn = knn_spec)
+)
```

Tune

```
R> set.seed(2026)

R> classification_results <- workflowsets::workflow_map(
+  classification_workflows,
+  fn = "tune_grid",
+  seed = 2026,
+  resamples = wdi_folds,
+  grid = neighbor_grid,
+  metrics = yardstick::metric_set(
+    yardstick::bal_accuracy,
+    yardstick::accuracy
+  ),
+  control = tune::control_grid(save_pred = TRUE),
+  verbose = FALSE
+)
```

`rank_results()` identifies the best neighbourhood size within each distance workflow and then ranks those workflows using balanced accuracy.

```
R> classification_ranking <- workflowsets::rank_results(
+  classification_results,
+  rank_metric = "bal_accuracy",
+  select_best = TRUE
+)
```

To examine the complete tuning profiles, we extract the resampling results for each workflow and collect the performance estimates for all evaluated values of the number of neighbours.

```
R> classification_metrics <- purrr::map_dfr(
+  classification_results$wflow_id,
+  function(wflow_id) {
+    workflowsets::extract_workflow_set_result(
+      classification_results,
+      wflow_id
```

```
+    ) |>
+      tune::collect_metrics() |>
+      dplyr::mutate(wflow_id = wflow_id)
+  }
+)
```

Distance	Neighbours	Balanced accuracy	Standard error	Rank
Unbiased mixed	9	0.806	0.020	1
Unbiased independent	9	0.791	0.021	2
Gower	5	0.756	0.016	3
Unbiased dependent	7	0.749	0.012	4
Euclidean	7	0.744	0.016	5

Table 6: Best nearest-neighbour configuration for each WDI distance, ranked by five-fold cross-validated balanced accuracy.

The best configurations for `u_mix` and `u_indep` have very similar balanced accuracies relative to their standard errors. These results describe performance under the present resampling design; they do not establish a general advantage for a response-aware or response-independent distance.

Figure 7 shows the complete tuning profiles rather than only the best configuration under each distance.

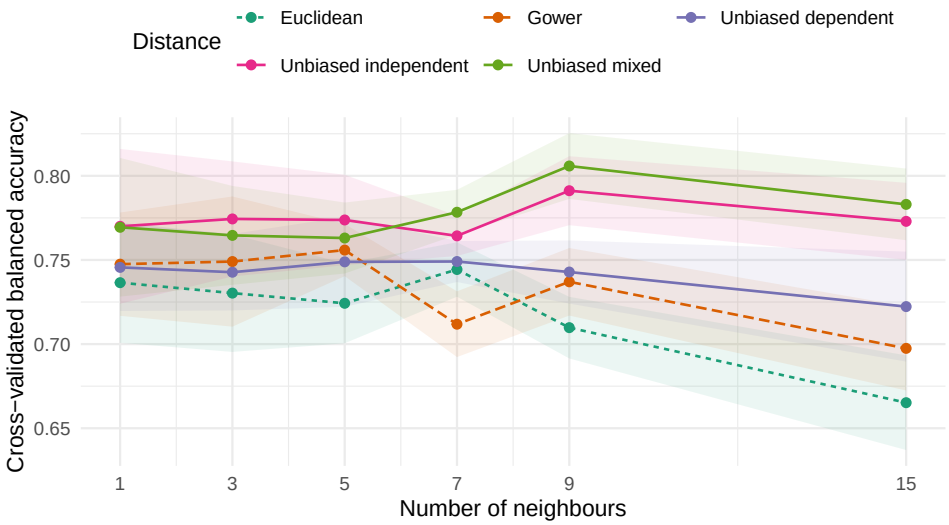


Figure 7: Five-fold cross-validated balanced accuracy for WDI income classification. Each curve tunes the number of neighbours under a different distance specification, with ribbons giving plus or minus one standard error.

Only after the distance specification and number of neighbours have been selected is the test set used. The `last_fit()` function from the **tune** package then refits the selected recipe and

nearest-neighbour model on the complete training set and evaluates the resulting workflow on the test countries.

Income classification: final evaluation

Select

```
R> selected_workflow_id <- classification_ranking |>
+  dplyr::filter(.metric == "bal_accuracy") |>
+  dplyr::slice_min(
+    rank,
+    n = 1,
+    with_ties = FALSE
+  ) |>
+  dplyr::pull(wflow_id)

R> selected_results <- workflowsets::extract_workflow_set_result(
+  classification_results,
+  selected_workflow_id
+)

R> best_neighbors <- tune::select_best(
+  selected_results,
+  metric = "bal_accuracy"
+)
```

Finalize

```
R> final_wdi_workflow <- workflowsets::extract_workflow(
+  classification_results,
+  selected_workflow_id
+) |>
+  tune::finalize_workflow(best_neighbors)
```

Evaluate

```
R> final_wdi_results <- tune::last_fit(
+  final_wdi_workflow,
+  split = wdi_split,
+  metrics = yardstick::metric_set(
+    yardstick::bal_accuracy,
+    yardstick::accuracy
+  )
+)

R> final_wdi_metrics <- tune::collect_metrics(final_wdi_results)

R> final_wdi_metrics
# A tibble: 2 x 4
  .metric      .estimator .estimate .config
  <chr>        <chr>          <dbl> <chr>
1 bal_accuracy macro          0.800 pre0_mod0_post0
2 accuracy     multiclass     0.717 pre0_mod0_post0
```

Cross-validation selected the Unbiased mixed distance with 9 neighbours. On the test countries, the selected workflow attained balanced accuracy 0.800 and ordinary accuracy 0.717. Balanced accuracy averages class-specific performance across the four groups, whereas ordinary accuracy is the overall proportion of correctly classified countries. Given the modest test-set class counts, these values complete the workflow illustration rather than establish a general performance ranking.

```
R> wdi_test_predictions <- tune::collect_predictions(
+  final_wdi_results
```

```
+)

R> wdi_confusion <- yardstick::conf_mat(
+  wdi_test_predictions,
+  truth = income,
+  estimate = .pred_class
+)
```

Prediction	High	Low	Lower middle	Upper middle
High	15	0	0	2
Low	0	4	4	0
Lower middle	0	2	6	2
Upper middle	1	0	2	8

Table 7: Test-set WDI income-group confusion matrix. Rows are predicted classes and columns are observed classes.

Together, the WDI applications illustrate how the fitted-distance interface supports different learning tasks. In clustering, `step_mdist()` supplies a pairwise representation that can be combined with alternative clustering methods. In classification, it supplies new-to-training distances so that distance specification and neighbourhood size can be evaluated under a common resampling design, with all fitted preprocessing—including any response-aware categorical dissimilarities—restricted to the appropriate analysis sets.

7. Conclusion

The **manydist** package implements the general mixed-variable distance framework of van de Velden *et al.* (2026), providing a flexible way to construct distances by combining numerical and categorical contributions, with purely numerical and purely categorical settings arising as special cases. In addition, the package includes several established and recently proposed distance specifications that do not fall directly within this framework. Beyond distance construction, **manydist** provides leave-one-variable-out diagnostics for examining variable-specific contributions to the overall distance, together with tools for comparing such diagnostics across alternative distance specifications. It also provides interfaces to distance-based clustering and nearest-neighbour methods and integrates these procedures into resampling and tuning workflows. Because the fitted `MDist` object retains the preprocessing and distance components estimated from the training data, the same construction can be applied to new observations without re-estimation. The `step_mdist()` interface fits these components within each resample, preventing held-out observations from influencing the learned distance.

Two applications illustrated the interface. The Palmer penguins data showed how scale and association shape the distance geometry, and how leave-one-variable-out diagnostics and pairwise comparisons of candidate distances make the choice of distance an explicit, inspectable part of an analysis rather than a fixed preprocessing decision. The World Development Indicators snapshot then placed the distance inside **tidymodels** pipelines: as a component

compared through clustering agreement and separation, and as a tuned choice evaluated jointly with neighbourhood size under a common resampling design. In both settings the diagnostics indicate whether a substantive conclusion is robust to the choice of distance or depends on it. **The tidymodels integration eliminates the need to write and maintain manual loops by automating repetitive model-fitting and evaluation tasks within a consistent framework. This makes the code cleaner and easier to reproduce, while simplifying preprocessing, parameter tuning, resampling, and performance comparison across multiple models.**

The framework has deliberate limits. An unbiased distance treats all variables equivalently by construction; it is a neutral starting point, not a claim of optimality, and a variable can and should be reweighted when subject-matter knowledge justifies it. The component dissimilarities use associations within the numerical and categorical blocks but not across them, so cross-type interactions are not represented. **Response-aware constructions are likewise confined to the categorical block, so a response variable cannot yet shape the numerical contributions.** The diagnostics are sensitivity analyses rather than model-selection rules: they show how much an analysis rests on the choice of distance, but they do not, on their own, select one.

These boundaries also mark the natural directions for further work: cross-type interaction-aware distances, response-aware constructions for the numerical block, and scaling the commensurable distances to larger problems through the compiled and parallel back-ends already available for homogeneous data. By making distance construction, diagnosis, and use a single interface, **manydist** aims to make the choice of a mixed-variable distance a transparent and reproducible step in distance-based learning.

Computational details

The results in this article were obtained using R 4.5.2 with **manydist** 0.5.2, **tidymodels** 1.4.1, **dplyr** 1.2.0 and **palmerpenguins** 0.1.1. R itself and all packages used are available from CRAN at https://CRAN.R-project.org/. The development version of **manydist** and its documentation are hosted at https://alfonsoiodicede.github.io/manydist_package/.

References

Ahmad A, Dey L (2007). "A k-Mean Clustering Algorithm for Mixed Numeric and Categorical Data." *Data & Knowledge Engineering*, **63**(2), 503–527.

Borg I, Groenen PJF (2005). *Modern Multidimensional Scaling: Theory and Applications*. Springer Series in Statistics, 2 edition. Springer, New York. doi:10.1007/0-387-28981-X.

Budiaji W (2022). *kmed: Distance-Based K-Medians/Medoids Clustering and Mixed Variable Distances*. URL https://CRAN.R-project.org/package=kmed.

D'Orazio M, Bridi P, Cecile F (2023). *StatMatch: Statistical Matching or Data Fusion*. URL https://CRAN.R-project.org/package=StatMatch.

Eckert A (2025). *parallelDist: Parallel Distance Matrix Computation Using Multiple Threads*. doi:10.32614/CRAN.package.parallelDist. URL https://CRAN.R-project.org/package=parallelDist.

Foss A, Markatou M, Ray B, Heching A (2016). "A Semiparametric Method for Clustering Mixed Data." *Machine Learning*, **105**, 419–458.

Foss AH, Markatou M (2018). "kamila: Clustering Mixed-Type Data in R and Hadoop." *Journal of Statistical Software*, **83**(13), 1–45. doi:10.18637/jss.v083.i13.

Ghashti JS, Thompson JRJ (2024). "Mixed-Type Distance Shrinkage and Selection for Clustering via Kernel Metric Learning." *Journal of Classification*, **42**(2), 311–334. doi:10.1007/s00357-024-09493-z.

Gower JC (1971). "A General Coefficient of Similarity and Some of Its Properties." *Biometrics*, **27**(4), 857–874. doi:10.2307/2528823.

Hahsler M, Grün B, Hornik K (2023). *arules: Mining Association Rules and Frequent Itemsets*. URL https://CRAN.R-project.org/package=arules.

Hennig C, Liao TF (2013). "How to Find an Appropriate Clustering for Mixed-Type Variables with Application to Socio-Economic Stratification." *Journal of the Royal Statistical Society Series C: Applied Statistics*, **62**(3), 309–369.

Huang Z (1998). "Extensions to the k-Means Algorithm for Clustering Large Data Sets with Categorical Variables." *Data Mining and Knowledge Discovery*, **2**(3), 283–304. doi:10.1023/A:1009769707641.

Iodice D'Enza A, Markos A, van de Velden M, Cavicchia C (2026). *manydist: Distance-Based Learning for Mixed-Type Data*. R package version 0.5.1, URL https://cran.r-project.org/web/packages/manydist.

Kuhn M, Wickham H (2020). *Tidymodels: a collection of packages for modeling and machine learning using tidyverse principles*. URL https://www.tidymodels.org.

Laliberté E, Legendre P, Shipley B (2014). *FD: Measuring Functional Diversity from Multiple Traits, and Other Tools for Functional Ecology*. R package version 1.0-12.3, URL https://CRAN.R-project.org/package=FD.

Le SQ, Ho TB (2005). "An Association-Based Dissimilarity Measure for Categorical Data." *Pattern Recognition Letters*, **26**(16), 2549–2557.

Liu P, Yuan H, Ning Y, Chakraborty B, Liu N, Peres MA (2024). "A modified and weighted Gower distance-based clustering analysis for mixed type data: a simulation and empirical analyses." *BMC Medical Research Methodology*, **24**(1), 305.

Maechler M, Rousseeuw P, Struyf A, Hubert M, Hornik K (2025). *cluster: Finding Groups in Data*. CRAN. R package version 2.1.7, URL https://CRAN.R-project.org/package=cluster.

Markos A, D'Enza AI, van de Velden M (2019). "Beyond Tandem Analysis: Joint Dimension Reduction and Clustering in R." *Journal of Statistical Software*, **91**(10), 1–24. doi:10.18637/jss.v091.i10.

Meyer D, Buchta C (2023). *proxy: Distance and Similarity Measures*. doi:10.32614/CRAN.package.proxy. R package version 0.4-27, URL https://CRAN.R-project.org/package=proxy.

Mousavi E, Sehhati M (2023). "A generalized multi-aspect distance metric for mixed-type data clustering." *Pattern Recognition*, **138**, 109353. doi:10.1016/j.patcog.2023.109353.

Papadakis M (2025). *Rfast: A Collection of Efficient and Extremely Fast R Functions*. R package version 2.1.5.2, URL https://CRAN.R-project.org/package=Rfast.

R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

Sulc Z, Cibulkova J, Rezankova H (2025). *nomclust: Hierarchical Cluster Analysis of Nominal Data*. URL https://CRAN.R-project.org/package=nomclust.

Szepannek G (2018). "clustMixType: User-Friendly Clustering of Mixed-Type Data in R." *The R Journal*, pp. 200–208. doi:10.32614/RJ-2018-048.

The World Bank (2022). *World Development Indicators*. The World Bank, Washington, DC. Database. Accessed 2026-07-29, URL https://databank.worldbank.org/source/world-development-indicators.

Thompson JRJ, Ghashti JS (2025). *kdml: Kernel Distance Metric Learning for Mixed-Type Data*. doi:10.32614/CRAN.package.kdml. R package version 1.1.1, URL https://CRAN.R-project.org/package=kdml.

van de Velden M, Iodice D'Enza A, Markos A, Cavicchia C (2024). "A General Framework for Implementing Distances for Categorical Variables." *Pattern Recognition*, **153**, 110547. doi:10.1016/j.patcog.2024.110547.

van de Velden M, Iodice D'Enza A, Markos A, Cavicchia C (2026). "Unbiased Mixed-Variable Distance." *Journal of Computational and Graphical Statistics*. doi:10.1080/10618600.2026.2680181.

van der Loo MPJ (2025). *gower: Gower's Distance*. CRAN. R package version 1.0.2, URL https://CRAN.R-project.org/package=gower.

Weihs C, Ligges U, Luebke K, Raabe N (2005). "klaR Analyzing German Business Cycles." In D Baier, R Decker, L Schmidt-Thieme (eds.), *Data Analysis and Decision Support*, pp. 335–343. Springer-Verlag, Berlin.

Affiliation:

Alfonso Iodice D'Enza
Department of Political Sciences
University of Naples Federico II
Via Leopoldo Rodinò 22
80138 Naples, Italy
E-mail: iodicede@unina.it

Michel van de Velden
Econometric Institute, Erasmus School of Economics
Erasmus University Rotterdam
Burgemeester Oudlaan 50
3062 PA Rotterdam, The Netherlands
E-mail: vandevelden@ese.eur.nl

Angelos Markos
Department of Primary Education, School of Education
Democritus University of Thrace
Nea Chili
68132 Alexandroupolis, Greece
E-mail: amarkos@eled.duth.gr

Carlo Cavicchia
Econometric Institute, Erasmus School of Economics
Erasmus University Rotterdam
Burgemeester Oudlaan 50
3062 PA Rotterdam, The Netherlands
E-mail: cavicchia@ese.eur.nl

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd